

□ Ejercicio Python

Agenda de Contactos con Menú de Consola

Estructuras de datos · CRUD · Menú interactivo

1. Contexto

En este ejercicio vas a construir una aplicación de consola en Python para gestionar una agenda de contactos. El programa debe almacenar los contactos en memoria usando una lista de diccionarios y ofrecer al usuario un menú interactivo para realizar todas las operaciones básicas.

2. Objetivos de aprendizaje

- Trabajar con listas de diccionarios como estructura de datos tabular.
- Implementar un menú de consola con bucle principal y control de flujo.
- Aplicar las operaciones CRUD sobre una colección en memoria.
- Comprender la mutabilidad de los diccionarios en Python.
- Usar list comprehensions y generadores para búsquedas eficientes.

3. Estructura de datos requerida

Cada contacto debe representarse como un diccionario con las siguientes claves:

```
{ "id": int, "nombre": str, "telefono": str, "email": str, "grupo": str }
```

El conjunto de contactos debe almacenarse en una lista Python ordinaria, sin usar bases de datos ni archivos externos.

4. Funcionalidades requeridas

Operación	Función Python	Acción sobre la lista
CREATE - Agregar	agregar_contacto()	list.append(dict)
READ - Mostrar	mostrar_todos()	enumerate() + for
READ - Buscar	buscar_contacto()	list comprehension con filtro
UPDATE - Editar	editar_contacto()	mutabilidad de dicts
DELETE - Eliminar	eliminar_contacto()	list.remove(objeto)

5. Menú de consola

El programa debe mostrar el siguiente menú al iniciarse y volver a él después de cada operación:

```
=====
AGENDA DE CONTACTOS |  Menú
=====
1.  Mostrar todos los contactos
2.  Buscar por nombre
3.  Agregar contacto
4.  Editar contacto
5.  Eliminar contacto
0.  Salir
=====
```

6. Reglas de negocio

- No se puede agregar un contacto con nombre vacío.
- No se pueden tener dos contactos con el mismo nombre (comparación case-insensitive).
- La búsqueda debe ser parcial e insensible a mayúsculas (p.ej. "ana" debe encontrar "Ana García").
- Al editar, si el usuario deja un campo en blanco, debe conservarse el valor original.
- Antes de eliminar un contacto, solicitar confirmación al usuario (s/n).
- Si una búsqueda devuelve más de un resultado al editar o eliminar, mostrar la lista y pedir que seleccione uno.

7. Datos iniciales de prueba

Inicializá la agenda con al menos 3 contactos de ejemplo para poder probar el programa desde el primer momento sin necesidad de agregar datos manualmente.

8. Conceptos de Python involucrados

- **Listas y diccionarios:**

Acceso por clave, mutabilidad, `append()` y `remove()`.

- **List comprehension con filtro:**

```
[c for c in contactos if termino in c['nombre'].lower()]
```

- **enumerate():**

Para mostrar índices numerados al recorrer la lista.

- **Generador con next():**

Búsqueda que se detiene en el primer resultado sin recorrer toda la lista.

- **Operador or para valores por defecto:**

`contacto['nombre'] = nuevo or contacto['nombre']`

- **Diccionario de funciones:**

Mapear opciones del menú a callables para evitar cadenas largas de `if/elif`.

9. Criterios de evaluación

- ✓ El programa inicia sin errores y muestra el menú correctamente.
- ✓ Cada operación CRUD funciona según las reglas de negocio.
- ✓ La búsqueda es parcial e insensible a mayúsculas.
- ✓ Las validaciones (nombre vacío, duplicado, confirmación de eliminación) están implementadas.
- ✓ El código está organizado en funciones con nombres descriptivos.
- ✓ El menú regresa después de cada operación hasta que el usuario elige Salir.

10. Modalidad de entrega

- Un único archivo Python: `agenda_contactos.py`
- El archivo debe ejecutarse directamente con: `python3 agenda_contactos.py`
- No se deben usar librerías externas; solo módulos de la biblioteca estándar (p.ej. `os` para limpiar pantalla).